

BİTİRME PROJESİ TEKNİK DOKÜMANI

OptiSched Backend

Mimari, API, PostgreSQL, CP-SAT Algoritması ve İş Akışları

Dokümanın amacı

Bu belge, projenin backend tarafını sunum ve jüri hazırlığı için uçtan uca açıklar. Kodun mevcut davranışını, veri akışını, kısıt modelini, doğrulama mekanizmasını ve bilinen teknik sınırları birbirinden ayırır.

Hazırlayan: Mehmet Akif Duran

Haziran 2026

1. Yönetici Özeti

30 saniyelik anlatım

OptiSched, üniversite ders programlarını PostgreSQL'de tutulan kurallara göre üreten bir FastAPI servsidir. OR-Tools CP-SAT çözücüsü önce ortak dersleri, ardından 12 mühendislik bölümünü sırayla planlar. Öğretim üyesi, dönem/şube, süre, bölünmüş ders ve zaman kısıtları çözüm sırasında uygulanır; sonuçlar bağımsız doğrulayıcıdan geçirilerek JSONB program belgeleri halinde saklanır.

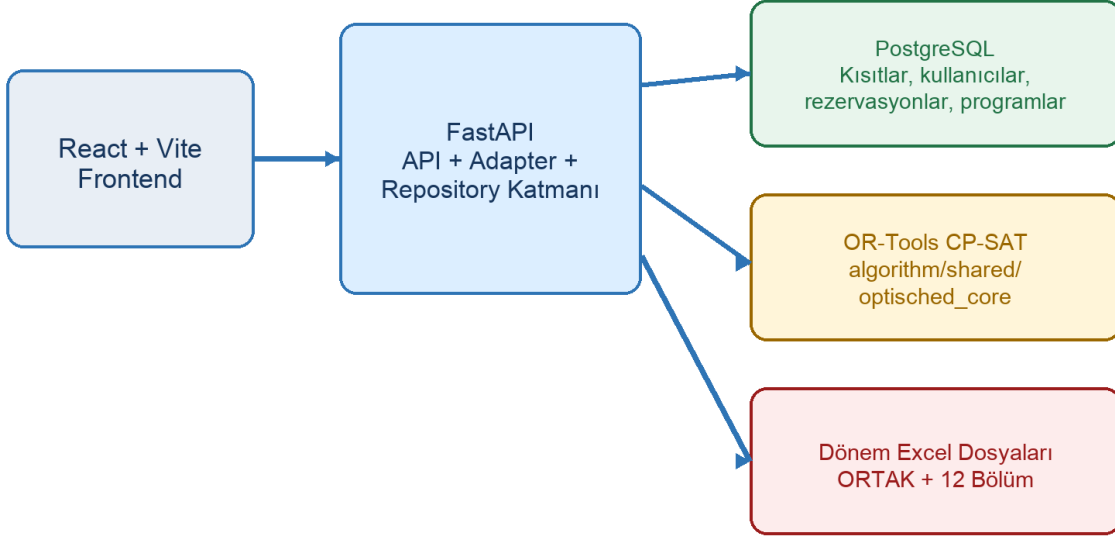
Backendin dört ana sorumluluğu vardır:

- Frontend'e kimlik doğrulama, ders, program, kısıt ve rezervasyon API'leri sunmak.
- Düzenlenebilir algoritma kısıtlarını PostgreSQL'den okumak ve yine PostgreSQL'e kaydetmek.
- Excel ve veritabanı verilerini OR-Tools çözücüsünün beklediği tablo biçimine dönüştürmek.
- Üretilen programları doğrulamak, frontend modeline çevirmek ve dönem/bölüm bazında kalıcılaştırmak.

Katman	Teknoloji	Temel rol
HTTP/API	FastAPI 0.115	Endpoint, CORS, hata yanıtı ve bağımlılık yönetimi
Şema	Pydantic 2	İstek/yanıt doğrulama ve tip sözleşmesi
Veri	PostgreSQL + psycopg2	Kullanıcı, ders, kısıt, rezervasyon ve program saklama
Algoritma	OR-Tools CP-SAT 9.12	Uygun ders-zaman yerleşimi ve amaç fonksiyonu
Dönüşüm	pandas + openpyxl	Excel okuma, veri normalizasyonu ve çıktı işleme

2. Sistem Mimarisi

OptiSched Backend Mimarisi



Şekil 1. OptiSched backend bileşenleri ve temel veri yönleri.

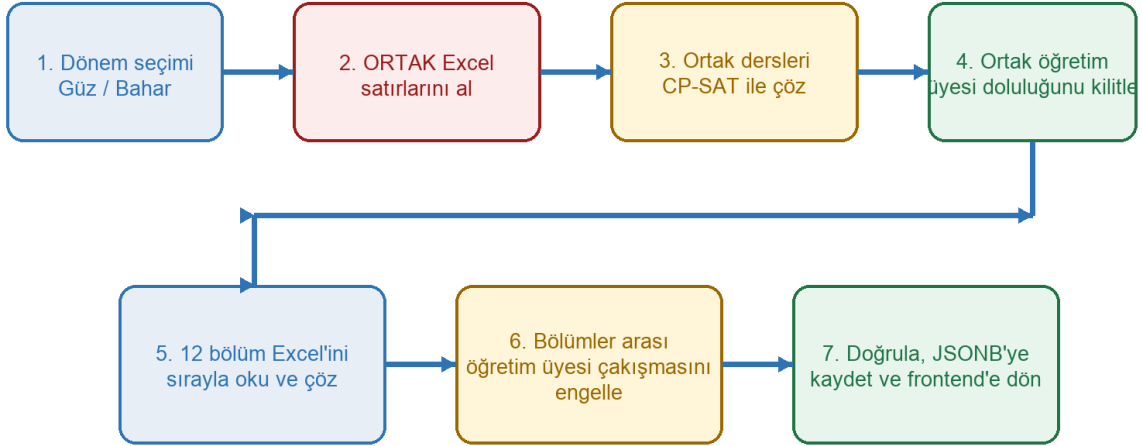
FastAPI, frontend ile veri kaynakları arasında orkestrasyon katmanıdır. Algoritma doğrudan HTTP ya da PostgreSQL bilmez; pandas DataFrame ve yapılandırma nesneleri üzerinden çalışır. Bu ayrım, çözücünün terminal uygulamasında ve web backendinde ortak kullanılmasını sağlar.

2.1 Kaynak Kod Yerleşimi

Konum	Sorumluluk
backend/app/main.py	FastAPI uygulaması ve bütün HTTP endpointleri
backend/app/models.py	Pydantic istek/yanıt modelleri
backend/app/auth.py	Parola doğrulama ve bearer token üretimi
backend/app/*_repository.py	PostgreSQL sorguları ve kalıcılık
backend/app/scheduler_adapter.py	DB/Excel verisini algoritmaya bağlayan adaptör
algorithm/shared/optisched_core/	CP-SAT model, doğrulama, blok sıkıştırma ve sınıf atama yardımcıları
database/migrations/	Kısıt, program ve rezervasyon tablolarını ekleyen migrationlar
real_deal_database/	Güz/Bahar bölüm Excel dosyaları

3. Uçtan Uca Program Üretim Akışı

Tek Tıklamada İki Aşamalı Program Üretimi



Şekil 2. Kullanıcı açısından tek işlem, backend açısından iki aşamalı fakülte çözümü.

3.1 Aşama 1: Ortak Dersler

1. Frontend, seçilen dönemin ORTAK Excel satırlarını `common\_rows` alanıyla gönderir.
2. Backend bu satırları teori/laboratuvar saatlerine göre ders parçalarına dönüştürür.
3. Ortak ders çözümünde dönem-şube çakışması kapatılır; öğretim üyesi, zaman ve bağlantılı ders kuralları uygulanır.
4. Ortak ders çözümü başarısızsa bölüm aşamasına geçilmez ve açıklayıcı hata döndürülür.

3.2 Aşama 2: Bölümler

5. Seçilen dönemin 12 bölüm Excel dosyası sırayla okunur: BİL, EEM, END, CE, BME, MAK, AI, CSE, BENG, EEE, IND ve ME.
6. Ortak derslerde dolu olan öğretim üyesi zamanları bölüm çözümlerine yasak zaman olarak aktarılır.
7. Her bölüm çözüldükçe yeni öğretim üyesi dolulukları sonraki bölümlere taşınır.
8. Son aşamada birleşik program üzerinde bölümler arası öğretim üyesi çakışması tekrar kontrol edilir.
9. Ortak ve bölüm programları `published\_schedule\_documents` tablosuna ayrı JSONB belgeleri olarak kaydedilir.

Dönem seçiminin etkisi

`fall` yalnızca tek dönem numaralı; `spring` yalnızca çift dönem numaralı dersleri tekil çalıştırma akışında filtreler. Fakülte akışında ise doğru `GUZ` veya `BAHAR` Excel dizini seçilir. Akademik yıl görünürde kaldırılmış olsa da backend program anahtarını şu anda `0000-0000|fall/spring` biçiminde üretmektedir.

4. API Sözleşmesi

Metot	Endpoint	Kimlik	Amaç
GET	/api/health	Hayır	Servis sağlık kontrolü
POST	/api/auth/login	Hayır	Kullanıcı doğrular, token döndürür
GET	/api/auth/me	Evet	Aktif hesabı döndürür
POST	/api/auth/logout	Hayır	204 döndürür; token sunucuda tutulmaz
GET	/api/courses	Hayır	Ders ve bölüm eşleşmelerini listeler
GET	/api/rooms	Hayır	Derslikleri kapasite ve türle listeler
GET	/api/rooms/availability	Hayır	Tarih/saat için uygunluk hesaplar
GET	/api/reservations	Evet	Rezervasyonları listeler
POST	/api/reservations	Evet	Çakışma kontrollü rezervasyon oluşturur
DELETE	/api/reservations/{id}	Evet	Yetkiye göre rezervasyon siler
GET	/api/scheduler/inputs	Hayır	DB'den algoritma ders girdilerini üretir
POST	/api/scheduler/inputs	Evet	Tek ders girdisi ekler/günceller
GET/PUT	/api/scheduler/common	PUT: Evet	Ortak programı okur/yazar
GET/PUT	/api/scheduler/dept	PUT: Evet	Bölüm programlarını okur/yazar
GET/PUT	/api/scheduler/linked	PUT: Evet	Bağlantılı ders gruplarını yönetir
GET/PUT	/api/scheduler/constraint-bundle	Hayır	Kısıt paketini okur/yazar
POST	/api/scheduler/run	Hayır	Tek kapsamlı çözücü akışını çalıştırır
POST	/api/scheduler/run-university	Opsiyonel	Ortak + 12 bölüm akışını çalıştırır ve kaydeder

Sunumda vurgulanacak nokta

Frontend'in algoritma kurallarını doğrudan çözücüye göndermesi yerine, kısıt paketi önce PostgreSQL'e yazılır; backend çalıştırma anında paketi yeniden veritabanından okur. Böylece kullanılan kural seti kayıt altındadır.

5. Veri Modelleri ve Dönüşümler

Model	Önemli alanlar	Kullanım
AlgorithmInput	kod, ad, haftalık saat, dönem, şube, öğretim üyesi, tür	DB ders bilgisinin çözücü girdisine taşınması
ConstraintBundle	settings, systemRules, constraints, linkedGroups, splits	Bir dönem ve bölüm için bütün algoritma ayarları
ScheduledCourse	gün, başlangıç/bitiş, şube, sınıf, bölüm, dinleyici bölümler	Frontend'in gösterdiği program satırı
UniversitySchedulerRunRequest	term, courses, common_rows, view_department, common_only	Fakülte çapında çalıştırma isteği
ScheduleStats	süre, yerleşen görev, hata, uyarı, öğretim üyesi saatleri	Çalıştırma özeti ve gözlemlenebilirlik
ReservationWrite	derslik, tarih, saatler, ders, öğretim üyesi	Rezervasyon oluşturma isteği

5.1 Excel Normalizasyonu

- `Ders Kodu` büyük harfe ve boşluksuz biçime getirilir.
- `Şube` sayısallaştırılır; eksikse 1 kabul edilir.
- `t\_hour` ve `l\_hour` toplamı haftalık süreyi belirler; bunlar yoksa `Kredi` kullanılır.
- 4 saat 2+2, 5 saat 2+3, 6 saat 3+3 olarak bölünebilir; DB'deki özel split tercihi önceliklidir.
- Öğrenci sayısında önce `Sınıf Mevcudu`, sonra `Kontenjan`, son olarak 0 kullanılır.
- Bitirme Projesi I/II ve Graduation Project I/II adları girdi, çıktı ve kayıt aşamalarında filtrelendir.

5.2 Frontend Çıktısına Dönüşüm

Çözücü her ders saati için ayrı satır üretir. `compress\_schedule\_blocks` ardışık saatleri tek ders bloğunda birleştirir. Adaptör gün ve saat indekslerini `Mon`/`09:00` biçimine çevirir, öğretim üyesi yoksa `Atanmamış`, sınıf yoksa varsayılan `TBA` değerini kullanır.

6. PostgreSQL Tasarımı

Mevcut üniversite şeması dersler, şubeler, öğretim üyeleri, bölümler, kullanıcılar ve derslikler için ana kaynaktır. Proje migrationları bu şemayı silmeden yalnızca çizelgeleme ve rezervasyon tablolarını ekler.

Tablo	Görev	Önemli ilişki
academic_periods	Akademik yıl + güz/bahar anahtarı	year_label + term benzersiz
scheduler_constraint_sets	Takvim, süre, ağırlık ve hariç prefix ayarları	period + department benzersiz
scheduler_system_rules	Açılıp kapanabilen kural bayrakları	constraint_set'e bağlı
scheduler_time_constraints	Hoca müsait değil / ders sabit / ders yasak	hedef + gün + saat
scheduler_course_split_preferences	Dersin haftalık saat bölme tercihi	ders kodu + sıra
linked_course_groups	Birlikte başlayan ders grubu üst kaydı	döneme bağlı UUID
linked_group_courses	Gruba bağlı ders kodları	group_id + course_code
published_schedule_documents	Ortak ve bölüm programlarını JSONB saklar	period + department benzersiz
classroom_reservations	Tarihli derslik rezervasyonları	derslik, tarih, text[] saat

`HAVUZ` kapsamı

Ortak dersler veritabanında özel bir bölüm kodu olan `HAVUZ` ile temsil edilir. Diğer programlar gerçek bölüm kodlarıyla saklanır.

7. Kısıt Modeli ve CP-SAT

Her ders parçası, gün ve saat için bir Boolean karar değişkeniyle modellenir. Başlangıç değişkenleri, dersin tek günde ardışık saatlere yerleşmesini sağlar. OR-Tools CP-SAT uygun bir çözüm ararken zorunlu kuralları ihlal edemez; yumuşak hedefleri ise ağırlıklı maliyet olarak minimize eder.

7.1 Zorunlu Kurallar

Kod	Kural	Açıklama
H1	Haftalık saat	Her ders parçası tam olarak beklenen saat kadar yerleşir.
H2	Tek gün + ardışıklık	Bir parça aynı gün içinde kesintisiz saatlere yerleşir.
H3	Bölünmüş parçalar	Aynı dersin 2+2 gibi parçaları farklı günlerde olur.
H4	Öğretim üyesi	Aynı öğretim üyesi aynı anda iki farklı ders veremez.
H5	Zorunlu ders	Aynı dönem/şubenin zorunlu dersleri çakışamaz.
H6	Seçmeli-zorunlu	Aynı dönem/şubenin seçmeli zorunlu dersle çakışamaz.
H7	Kilitli yerleşim	Önceden sabitlenen ders saati korunur.
H8	Derslik	Atama yapılmışsa aynı derslik aynı anda iki derse verilemez.

7.2 Veritabanından Gelen Dinamik Kurallar

Alan	Çözücü etkisi
instructor_unavailable	Belirli öğretim üyesi için gün/saat yasaklanır.
course_fixed	Belirli ders gün/saatte zorunlu tutulur.
course_blocked	Belirli dersin gün/saatte yerleşmesi engellenir.
linkedGroups	Gruba bağlı derslerin başlangıçları eşitlenir.
splits	Dersin toplam saati için özel bölme deseni seçilir.
systemRules	Hoca, dönem, ardışık saat ve haftalık saat kontrolleri açılıp kapanır.

7.3 Yumuşak Hedefler

Hedef	Varsayılan ağırlık	Amaç
Yakın dönem çakışması	8	1-3, 2-4 gibi dönemlerin zorunlu ders çakışmasını azaltmak
Öğrenci boşluğu	0	Gün içindeki ders aralarını azaltmak; maliyetli olduğu için varsayılan kapalı
Öğle saati	2	12:00-13:00 kullanımını azaltmak
Hoca yük dengesi	1	Öğretim saatlerini günlere daha dengeli yaymak

Neden CP-SAT?

Problem; çakışmama, ardışıklık, sabitleme ve birlikte başlama gibi çok sayıda ayırık kararı aynı anda içerir. CP-SAT bu tür Boolean/tamsayı kısıt problemleri için uygundur ve süre sınırı içinde optimal ya da uygulanabilir çözüm döndürebilir.

8. Bağımsız Doğrulama Katmanı

Çözücünün sonuç üretmesi tek başına yeterli kabul edilmez. `validator.py`, CP-SAT modeline bakmadan somut gün/saat yerleşimlerini H1-H8 kurallarına göre yeniden kontrol eder. Bu yaklaşım modelleme veya son işleme hatalarını yakalamak için ikinci bir güvenlik katmanıdır.

- Tek bölüm doğrulaması: saat, ardışıklık, split, hoca, dönem/şube ve kilit kontrolü.
- Fakülte doğrulaması: bütün bölüm çıktıları birleştirilerek öğretim üyesi çakışması aranır.
- Uyarılar `ScheduleStats.warnings` alanıyla frontend'e döner.
- Çözücü boş sonuç döndürürse `solver-infeasible`, ortak aşama boşsa `common-infeasible` hatası üretilir.

9. Kimlik Doğrulama ve Yetki

Kullanıcı adıyla bulunan veritabanı kaydındaki `password\_hash`, girilen parolanın SHA-256 özetiyle sabit zamanlı karşılaştırılır. Başarılı girişte kullanıcı kimliği ve son kullanma zamanı içeren Base64 payload, `APP\_SECRET` ile HMAC-SHA256 imzalanır.

Konu	Mevcut uygulama	Değerlendirme
Token	Özel HMAC imzalı bearer token	JWT kütüphanesi kullanılmıyor; sunucu tarafı oturum kaydı yok
Süre	Varsayılan 28.800 saniye (8 saat)	`TOKEN_TTL_SECONDS` ile değiştirilebilir
Parola	SHA-256 hash karşılaştırması	Üretimde Argon2/bcrypt + salt tercih edilmelidir
Çıkış	204 No Content	Stateless token iptal listesi olmadığı için istemci tokeni siler
Roller	admin, dept_chair, coordinator, instructor, secretary, viewer	Silme işleminde rol kontrolü var; tüm endpointlerde tam RBAC yok

Üretim güvenliği

Varsayılan geliştirme secret'ı kullanılmamalı; uzun ve rastgele `APP\_SECRET` tanımlanmalı. Constraint bundle ve scheduler çalıştırma endpointlerinin bir kısmı şu an kimlik doğrulama istemediği için üretim öncesinde rol bazlı koruma eklenmelidir.

10. Rezervasyon Sistemi

- Derslikler kapasite ve tür bilgisiyle filtrelendir.
- Seçilen tarih ve saatler, hem manuel rezervasyonlarla hem yayınlanmış ders programıyla karşılaştırılır.
- PostgreSQL `text[]` çakışma operatörü (`&&`) aynı saatlerden en az birinin dolu olup olmadığını kontrol eder.
- Aynı derslik+tarih için transaction seviyesinde advisory lock alınır; eşzamanlı çift rezervasyon yarışı engellenir.
- Çakışmada HTTP 409, bulunmayan derslikte HTTP 404 döner.
- Admin ve bölüm başkanı her rezervasyonu; diğer roller yalnızca kendi rezervasyonunu silebilir.

Dönem eşleme ayrıntısı

Rezervasyon uygunluğu, tarihin ayına göre `spring` veya `fall` seçer ve yayınlanmış programı `0000-0000` yıl etiketi altında arar. Bu geçici yıl anahtarı ileride gerçek akademik yıl modeliyle değiştirilmelidir.

11. Sınıf Atama Durumu

`RoomAllocator`, kapasitesi yeterli en küçük uygun dersliği seçebilen ve aynı zaman diliminde derslik çalışmasını engelleyen bir bileşendir. Ancak mevcut web backend akışı bu bileşeni çağırılmamaktadır. Sonuç DataFrame'inde `Sınıf` alanı oluşmadığı için frontend modelinde oda varsayılan olarak `TBA` kalabilir.

Durum	Sonuç
Algoritma sınıf ataması çağırılmıyor	Program zamanı üretilir; fiziksel derslik garanti edilmez.
`TBA`	To Be Announced: sınıf henüz belirlenmedi anlamındadır.
Rezervasyon kontrolü	Yalnızca program satırında gerçek oda varsa doluluğu görebilir; `TBA` satırları yok sayılır.
Tam entegrasyon için	Derslik DataFrame'i DB'den okunmalı, blok sıkıştırmadan sonra `RoomAllocator.allocate` çağırılmalı ve H8 doğrulanmalıdır.

12. Hata Yönetimi ve Kalıcılık

Katman	Davranış
DB bağlantısı	Context manager başarılı işlemde commit, hatada rollback, her durumda close yapar.
DB genel hatası	FastAPI exception handler HTTP 503 ve hata mesajı döndürür.
Model doğrulama	Pydantic geçersiz dönem, saat, rol ve alanları 422 ile reddeder.
Program kaydı	Period+bölüm çakışmasında JSONB belge upsert edilir; updated_at yenilenir.
Bitirme projesi filtresi	Hem kaydetmeden önce hem okurken tekrar uygulanır.
Constraint kaydı	Paket bütün olarak değiştirilir; alt zaman/split/rule kayıtları transaction içinde yeniden yazılır.

12.1 Migration Stratejisi

Migrationlar eklemeli ve mümkün olduğunca idempotent tasarlanmıştır.

`003\_scheduler\_schema\_compatibility.sql`, aynı adlı eski tablolar farklı kolona sahipse onları `\_legacy` adına taşıyıp güncel şemayı kurar. Bu, eski veriyi silmeden isim çakışmasını çözmeyi amaçlar.

13. Testler ve Mevcut Doğrulama

Test grubu	Kapsam
test_scheduler_adapter.py	Bitirme projesi filtresi, yüklenen ortak satırlar, aşırı zorunlu yük normalizasyonu
test_solver_constraints.py	Hoca müsait değil, sabit ders, bağlantılı ders, ortak ders öğrenci doluluğu
test_constraint_repository.py	Kısıt paketinin PostgreSQL'e yazılıp geri okunması; `RUN_DB_TESTS=1` ile

Son bilinen uçtan uca doğrulama

Geliştirme sırasında fakülte akışı 137 ortak satır ve 12 bölümle çalıştırılmış; 649 program oturumu, 0 bitirme projesi ve 0 uyarı ürettiği gözlemlenmiştir. Bu sonuç çalışma anına aittir; veri veya kısıt değiştiğinde yeniden test edilmelidir.

Bu doküman hazırlanırken birim testleri yeniden çalıştırılmıştır: 7 test başarılıdır. PostgreSQL round-trip entegrasyon testi, `RUN\_DB\_TESTS=1` verilmediği için atlanmıştır.

14. Bilinen Sınırlar ve Teknik Borçlar

Öncelik	Konu	Öneri
Yüksek	Sınıf atama bağlı değil	'RoomAllocator' fakülte/web akışına entegre edilmeli; H8 gerçek çıktı üzerinde çalıştırılmalı.
Yüksek	Bazı yazma/çalıştırma endpointleri korumasız	Constraint ve scheduler endpointlerine rol bazlı yetki eklenmeli.
Orta	Bölüm verisi sunucu Excel'inden okunuyor	Frontend'den yüklenen tüm bölüm dosyaları backend'e aktarılmalı veya DB'ye import edilmelidir.
Orta	Geçici akademik yıl anahtarı	'0000-0000' yerine kullanıcı tarafından seçilen gerçek akademik dönem kullanılmalı.
Orta	SHA-256 parola saklama	Argon2id veya bcrypt, benzersiz salt ve parola yükseltme planı uygulanmalı.
Orta	Logout token iptal etmiyor	Kısa süreli access token + refresh token/denylist tasarımı değerlendirilmeli.
Düşük	Test kapsamı sınırlı	API integration, rezervasyon concurrency ve tüm fakülte regresyon testleri eklenmeli.
Düşük	503 yanıtında ham DB mesajı	İstemciye genel hata, log sistemine ayrıntı gönderilmeli.

15. Kurulum ve Çalıştırma

15.1 Ortam Değişkenleri

Değişken	Örnek	Amaç
DATABASE_URL	postgresql://user:pass@localhost:5432/optisched	PostgreSQL bağlantısı
FRONTEND_ORIGIN	http://127.0.0.1:5173	CORS izinli frontend
APP_SECRET	uzun-rastgele-secret	Token HMAC imzası
TOKEN_TTL_SECONDS	28800	Token ömrü

15.2 Backend

```
cd /Users/akifduran/Desktop/new_project
python3 -m venv .venv
source .venv/bin/activate
pip install -r backend/requirements.txt

export DATABASE_URL='postgresql://USER:PASSWORD@localhost:5432/optisched'
for migration in database/migrations/*.sql; do
  psql "$DATABASE_URL" -v ON_ERROR_STOP=1 -f "$migration"
done

PYTHONPATH=. uvicorn backend.app.main:app --reload --host 127.0.0.1 --port 8000
```

15.3 Frontend Bağlantısı

```
cd /Users/akifduran/Desktop/new_project/frontend-files
printf 'VITE_API_BASE_URL=http://127.0.0.1:8000/api\n' > .env
npm install
npm run dev
```

API dokümantasyonu backend çalışırken `http://127.0.0.1:8000/docs` adresindeki Swagger UI üzerinden incelenebilir.

16. Sunum İçin Hazır Anlatım

16.1 90 Saniyelik Teknik Konuşma

Konuşma metni

Backendimizi FastAPI ve PostgreSQL ile geliştirdik. Kullanıcı, Güz veya Bahar dönemini seçip ortak ders dosyasını yüklediğinde sistem tek komutla iki aşamalı çalışıyor. İlk aşamada ortak dersler OR-Tools CP-SAT ile planlanıyor. Bu aşamada oluşan öğretim üyesi dolulukları sabitleniyor. İkinci aşamada 12 mühendislik bölümünün programı sırayla oluşturuluyor ve her çözüm sonraki bölümlere çıkışma bilgisi aktarıyor.

Takvim, sabit/yasak saatler, bağlantılı dersler ve split tercihleri PostgreSQL'de tutuluyor; dolayısıyla algoritma tarayıcıdaki geçici state'e değil kayıtlı kurallara göre çalışıyor. Çözümünden sonra bağımsız validator saat, ardışıklık, öğretim üyesi ve dönem-şube kurallarını yeniden kontrol ediyor. Sonuçlar ortak ve bölüm bazında JSONB olarak saklanıp frontend'e gönderiliyor.

16.2 Olası Jüri Soruları

Soru	Kısa cevap
Neden iki aşama?	Ortak dersler tüm bölümleri etkilediği için önce tek kez yerleştirilir; dolulukları bölüm çözümlerine aktarılır.
Neden OR-Tools?	Problem çok sayıda ayırık karar ve çakışmama kuralı içerir; CP-SAT uygulanabilir çözümü sistematik arar.
Kurallar nerede?	Takvim, rule switch, zaman kısıtı, split ve linked group değerleri PostgreSQL'dedir.
Çözümün doğru olduğunu nasıl biliyorsunuz?	CP-SAT modelinden bağımsız validator H1-H8 kontrollerini somut çıktı üzerinde tekrar yapar.
Excel yüklemek neden gerekli?	Ortak ders satırları istekte gelir; bölüm satırları mevcut sürümde seçilen dönem klasöründeki sonucu Excel'lerinden okunur.
TBA nedir?	Ders saati oluşmuş ancak fiziksel sınıf atama bileşeni bu akışa bağlanmamış demektir.
Programlar nerede saklanıyor?	'published_schedule_documents.courses' JSONB alanında dönem ve bölüm bazında.
Aynı hoca iki bölümde çakışır mı?	Bölümler sırayla çözülürken hoca doluluğu aktarılır; ayrıca birleşik sonuçta global kontrol yapılır.

17. Sonuç

OptiSched backend; HTTP servisleri, PostgreSQL tabanlı kural yönetimi ve CP-SAT çizelgeleme çekirdeğini çalışan bir bütün halinde birleştirir. Mimarının güçlü tarafı, kısıtların kalıcı olması, ortak/bölüm programlarının tek çalıştırmada üretilmesi ve sonucun bağımsız doğrulamadan geçirilmesidir. Üretim seviyesine geçişte öncelikli konular sınıf atama entegrasyonu, bütün kritik endpointlerde rol tabanlı yetkilendirme ve yüklenen bölüm dosyalarının backend veri akışına tam bağlanmasıdır.